

Monte-Carlo Simulation for Probability of Snow on Thanksgiving

Brook Hamilton

hamilbro@oregonstate.edu

CS 475: Parallel Programming

Project 5

Professor Jesse Rosenzweig

September 30, 2026

Oregon State University - Cascades

Introduction:

This project investigates the probability that the first snow of the year in Bend, Oregon, will fall on Thanksgiving. By collecting historical data of the earliest and latest snowfalls of the year, combined with the mean first snowfall and an estimated standard deviation, the Monte Carlo simulation was used to find the probability of snow on the target day. Box-Muller was used to transform the input of two standard, evenly distributed random numbers that are fed into the simulation to predict the probability of snow. Using OpenMP to create multiple threads, the simulations were run for up to a million different years to get an accurate probability of snow on Thanksgiving in Bend.

By using a C++ program to find two pieces of information:

1. The probability that the first snow will be on Thanksgiving
2. The performance change/parallel fraction of the number of trials compared to thread count

Machine Details:

Machine: Lenovo LOQ AMD Ryzen 7 7435HS

Architecture: X86_64

CPU(s): 16

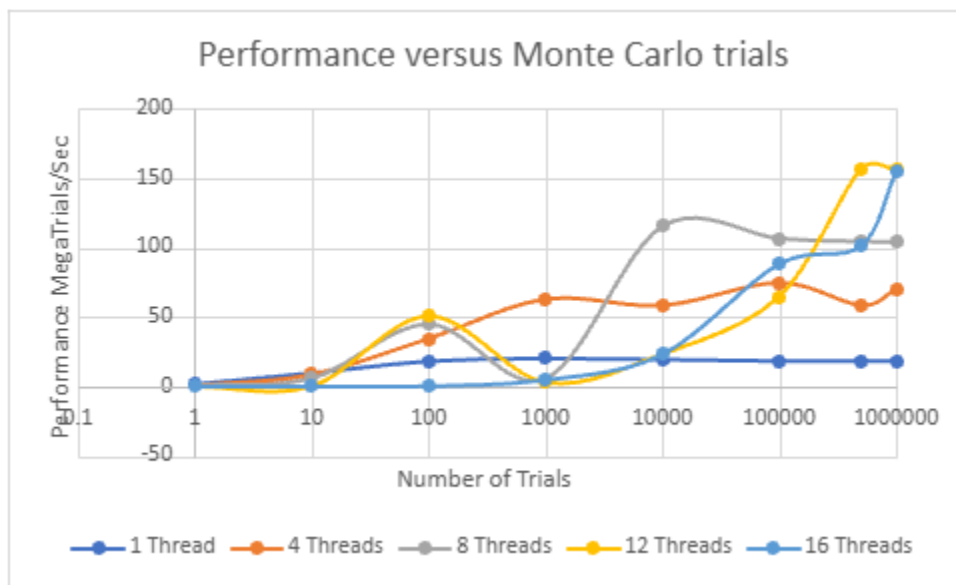
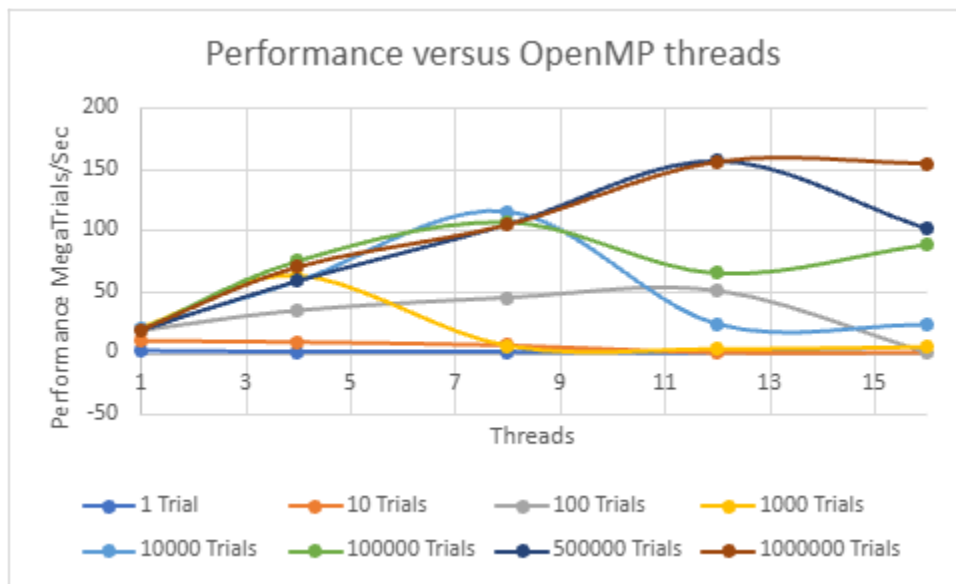
Thread(s) per core: 2

Core(s) per socket: 8

Results:

Performance as a function of NUMT and NUMTRIALS

NUMT	1 Trial	10 Trials	100 Trials	1000 Trials	10000 Trials	100000 Trials	500000 Trials	1000000 Trials
1	1.6	9.62	18.18	20.17	19.38	18.34	18.17	18.56
4	0.79	8.58	34.52	63	58.39	74.83	58.52	70.04
8	0.7	5.9	44.96	5.55	115.55	106.66	104.59	104.41
12	0.01	0.03	50.81	2.95	23.35	64.79	156.74	156.13
16	0	0.06	0.19	4.63	23.02	88.29	101.25	154.65



Speedup Efficiency:

$$S = 154.65/18.56$$

$$S = 8.33$$

Parallel Fraction:

$$F = 16/(16 - 1) * (8.33 - 1)/8.33$$

$$F = 16/15 * 7.33/8.33$$

$$F = 0.94$$

Conclusion:

Based on the probability achieved from 1000000 simulated trials, the probability that the first snow will be on Thanksgiving Day is 1.24%. This number stayed relatively consistent as the number of threads increased, although it predictably leveled as the number of trials grew, resulting in the most consistent and accurate estimate of 1.24% from the 1000000 trial tests. The consistency of results as trial numbers increases is predictable because there is more and more data to be compiled as the trial size grows. Despite the leveling out of the data, it more or less stays the same across all thread counts.

The results also show that an 8.33 speedup from 1 thread to 16 threads at 1000000 trials, with performance improving as the trial size increases, a subsequent plateau is also found which happens at around 12 threads starting at 500000 trials. As the efficiency decreases at around 8 threads it makes sense that a larger bottle neck is formed at around 12 as the resources available from memory and cpu are stretched thinner and thinner on the machine i am using. The 0.94 parallel fraction supports this as 94% of the work is being split up amongst all of the threads, but not quite all the work can be split, which also explains the drop off in efficiency as the thread count approaches my machine's maximum of 16 due to the 16 CPUs available.

While more threads emerge, the time spent on the parallel section of this problem shrinks, cutting the work required per thread to be less and less. Although as the time of work on the parallel section decreases the serial section of work remains constant and becomes more and more prominent while the overall speedup levels out.